Statistical Computing Midterm

Name: 莊明儒 Student ID: 111024513

Contents

1 Problem 1 **1**

1.1 a. 1

1.2 b. 7

2 Problem 2 **7**

2.1 a. 7

2.2 b. 14

1 Problem 1

1.1 a.

```
f = function(arg){
  x = arg[1]
  y = arg[2]
  return(ifelse(x^2+y^2 <= 1, (3/(2*pi))*sqrt(1-x^2-y^2), 0))
}
```

Figure 1 below shows how $f(x, y)$ looks like.

```
grid1 = seq(-1.5, 1.5, length.out = 100)
grid2 = expand.grid(grid1, grid1)
z = apply(grid2, 1, f)
par(mfrow = c(1,2), mar = c(3,3,1,1), mgp = c(1.6,.6,0))
persp(grid1, grid1, matrix(z, 100, 100), theta = 30, zlab = "f(x,y)", xlab = "x", ylab = "y")
quilt.plot(grid2[,1], grid2[,2], z, nx = 100, ny = 100, col = rev(heat.colors(1000)),
add.legend = F)
contour(grid1, grid1, matrix(z, 100, 100),
levels = round(quantile(z, seq(0.05,0.95,0.1)), 2), col = "blue", add = T)
```

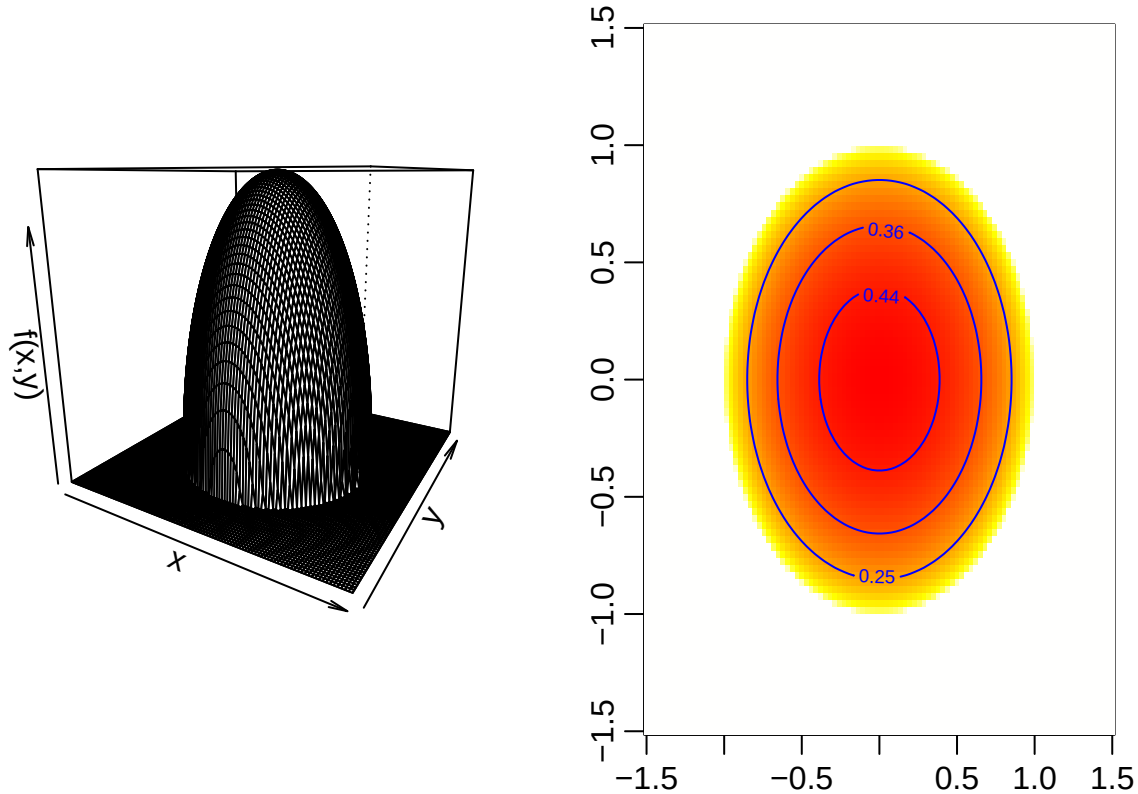

Figure 1: Visualization of $f(x, y)$

Since the support of $f(x, y)$ is contained in that of bivariate normal distribution $N(\mathbf{0}, \Sigma)$ with

$$\Sigma = \begin{pmatrix} \frac{2\pi}{9} & 0 \\ 0 & \frac{2\pi}{9}. \end{pmatrix}$$

I consider to generate the random sample via Metropolis-Hastings (MH) with proposal g, the density of $N(\mathbf{0}, \Sigma)$.

Algorithm

See appendix

Implementation

```
g = function(arg){
  x = arg[1]
  y = arg[2]
  dnorm(x, mean = 0, sd = sqrt(2*pi)/3)*dnorm(y, mean = 0, sd = sqrt(2*pi)/3)
}
```

```
mh = function(n){
  init = c(0, 0)
  x.m = matrix(NA, nrow = n, ncol = 2)
  x.m[1,] = init
```

```
  for(i in 2:n){
    u = rnorm(2, mean = 0, sd = sqrt(2*pi)/3)
    r = (f(u)/f(x.m[i-1,]))*(g(x.m[i-1,])/g(u))
    if(runif(1)<min(1,r)){
      x.m[i,] = u
    }else{
      x.m[i,] = x.m[i-1,]
    }
  }
  return(x.m)
}
```

```
n = 10^5
x.m1 = mh(n)
x.m2 = mh(n)
x.m3 = mh(n)
x.m4 = mh(n)
mc1234 = mcmc.list(mcmc(x.m1), mcmc(x.m2), mcmc(x.m3), mcmc(x.m4))
dump(c("mc1234"), "midterm_1.Rdata")
```

The change of Gelman-Rubin stastic along with the length of the Markov chains is shown in Figure 2. It seems that the Markov chains are convergent to the target distribution after the length is greater 92000. Hence, keep only the last 8000 sample points for the 4 Markov chains, resulting in 4 shortened Markov chains.

```
gelman.plot(mc1234)
```

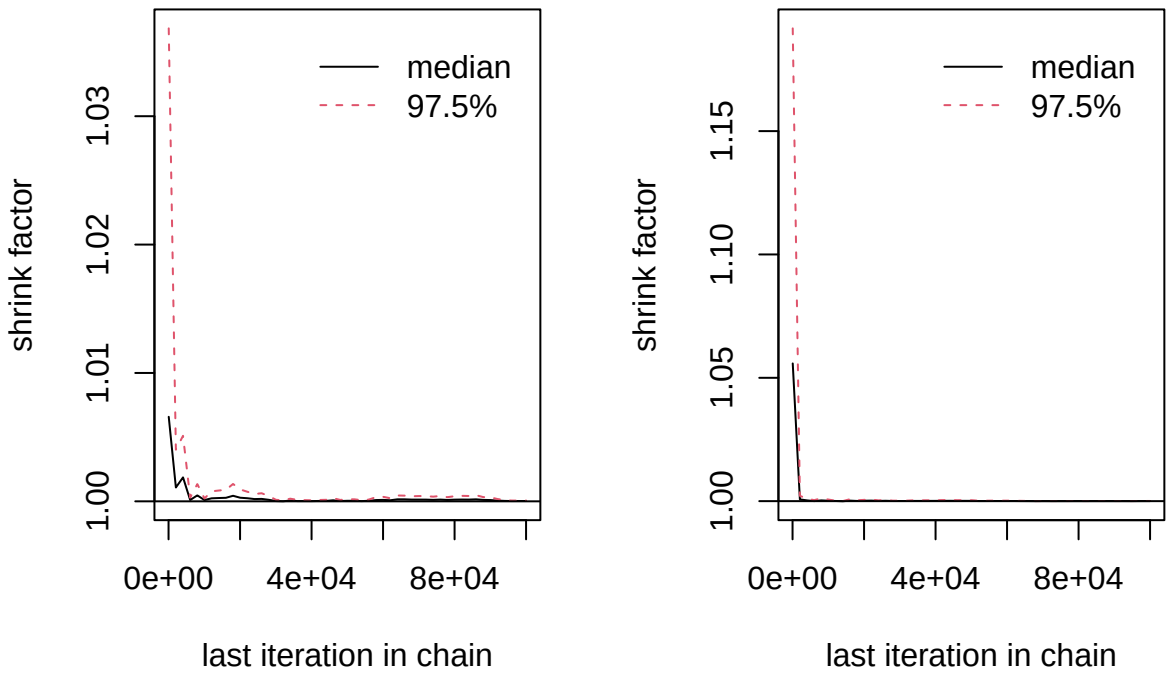

Figure 2: The change of Gelman-Rubin stastic along with the length of the Markov chains

After a few trials and errors, it turns out that by picking the sample points from each of the shortened Markov chains by every 32 ones, we can come up with an nearly i.i.d sample with size 1000, the final sample as required. Figure 3 shows the acf plots for the final sample. We can see that the autocorrelations of lag greater than or equal to 1 are not significant.

```
x.m = rbind(tail(mc1234[[1]], 8000)[32*(1:floor(8000)/32),],
            tail(mc1234[[2]], 8000)[32*(1:floor(8000)/32),],
            tail(mc1234[[3]], 8000)[32*(1:floor(8000)/32),],
            tail(mc1234[[4]], 8000)[32*(1:floor(8000)/32),])
x.m = x.m[sample(1000),]
acf(x.m)
```

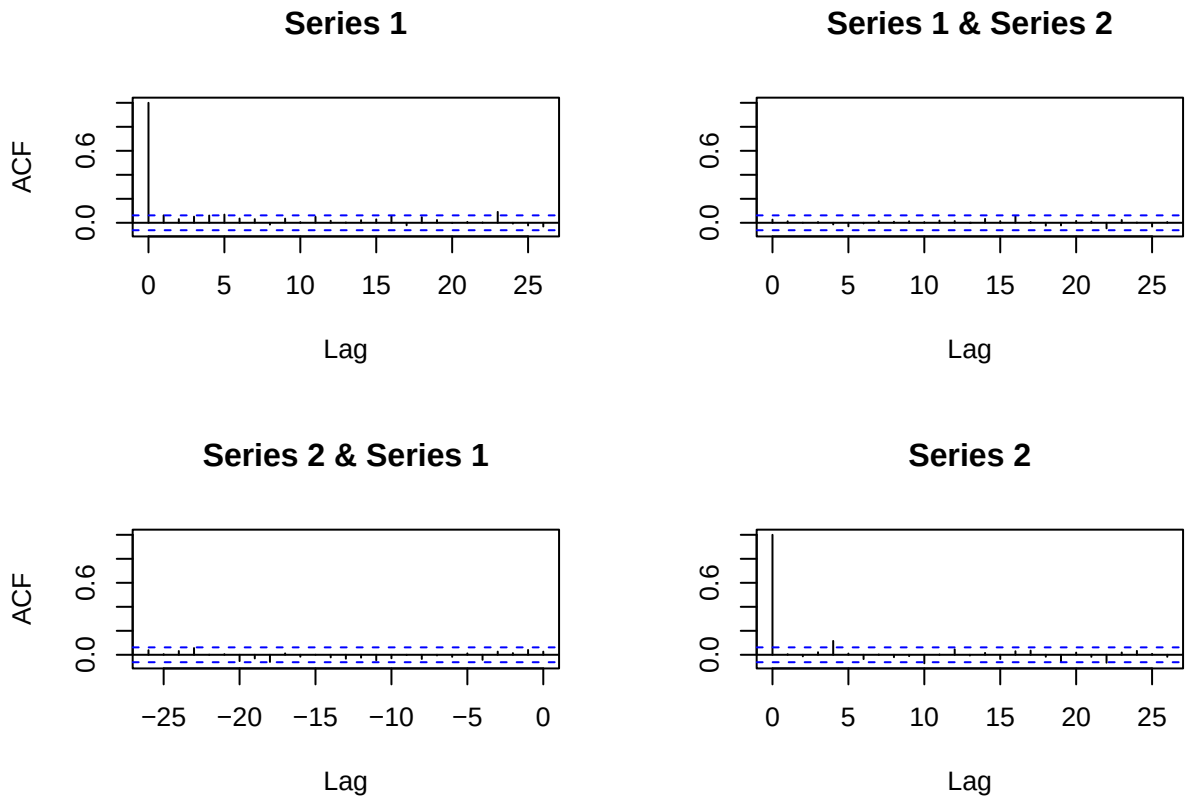

Figure 3: The acf plots for the final sample

Sampling result

Figure 4 shows the final sample by plotting the generated variables on the contour plot of $f(x, y)$. We can see that the distribution of the generated sample looks quite similar to $f(x, y)$.

```
par(mfrow=c(1,1), mar=c(3,3,1,1), mgp=c(1.6,0.6,0))
quilt.plot(grid2[,1], grid2[,2], z, nx = 100, ny = 100, col = rev(heat.colors(1000)),
add.legend = F)
points(x.m, pch = 16, col = rgb(0,0,0,0.4))
contour(grid1, grid1, matrix(z, 100, 100),
levels = round(quantile(z, seq(0.05,0.95,0.1)), 2), col = "blue", add = T)
```

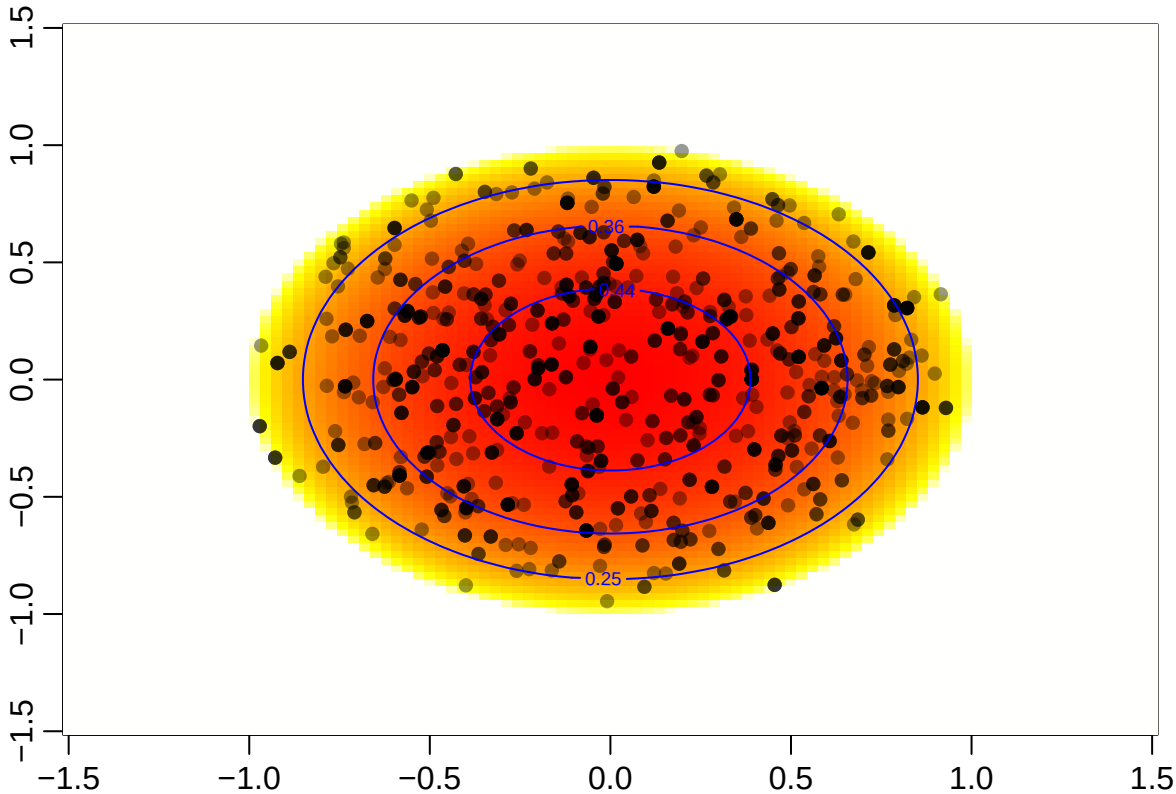

Figure 4: Sampling result in the view of $f(x, y)$

Figure 5 shows marginal distribution of the sample. We can see that the two marginal distributions looks quite similar to $f(x)$ and $f(y)$ respectively. For the derivation of $f(x)$ and $f(y)$, please see the appendix.

```
f.x = f.y = function(x){
  (2^(3/2))*(1-x^2)/pi
}
par(mfrow=c(1,2), mar=c(3,3,1,1), mgp=c(1.6,0.6,0))
hist(x.m[,1], 30, xlab = "y", main = "", probability = T)
curve(f.x, from = -1, to = 1, add = T, col = 2)
legend("topleft","f(x)", bty = "n", lty = 1, col =2)
hist(x.m[,2], 30, xlab = "y", main = "", probability = T)
curve(f.y, from = -1, to = 1, add = T, col = 2)
legend("topleft","f(y)", bty = "n", lty = 1, col = 2)
```

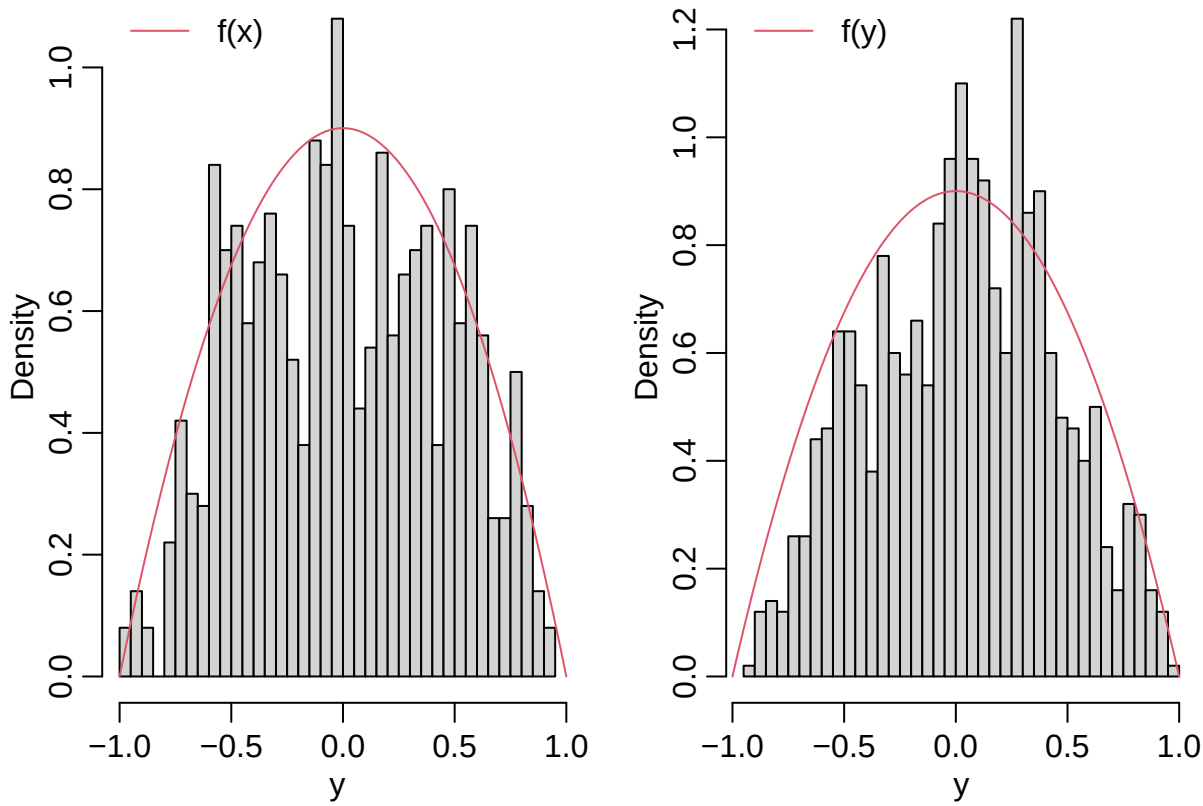

Figure 5: Sampling result in the view of $f(x)$ and $f(y)$

1.2 b.

For the algorithm and the formula of MC estimate and MC s.e., please see the appendix.

```
est = mean(x.m[,1]*x.m[,2])
se = sd(x.m[,1]*x.m[,2])/sqrt(1000)
c(est, se)
```

```
## [1] 0.005282202 0.004752489
```

It turns out the estimate is 0.0052822 and the s.e. is 0.0047525.

2 Problem 2

2.1 a.

```
f = function(arg){
  g(arg)
}
```

```
g = function(arg){
  x = arg[1]
  y = arg[2]
  y*(x+y)*(exp((-4)*(y^2)))*exp(-(x^2+2*x*y))*(1-exp(-(x^2+2*x*y)))^2
}
```

Figure 6 below shows how $f(x, y)$ looks like up to a constant c.

```
grid1 = seq(0.0001, 3, length.out = 100)
grid2 = expand.grid(grid1, grid1)
z = apply(grid2, 1, f)
par(mfrow = c(1,2), mar = c(3,3,1,1), mgp = c(1.6,.6,0))
persp(grid1, grid1, matrix(z, 100, 100), theta = 30, zlab = "f(x,y)", xlab = "x", ylab = "y")
quilt.plot(grid2[,1], grid2[,2], z, nx = 100, ny = 100, col = rev(heat.colors(1000)),
add.legend = F)
contour(grid1, grid1, matrix(z, 100, 100),
levels = round(quantile(z, seq(0.05,0.95,0.1)), 2), col = "blue", add = T)
```

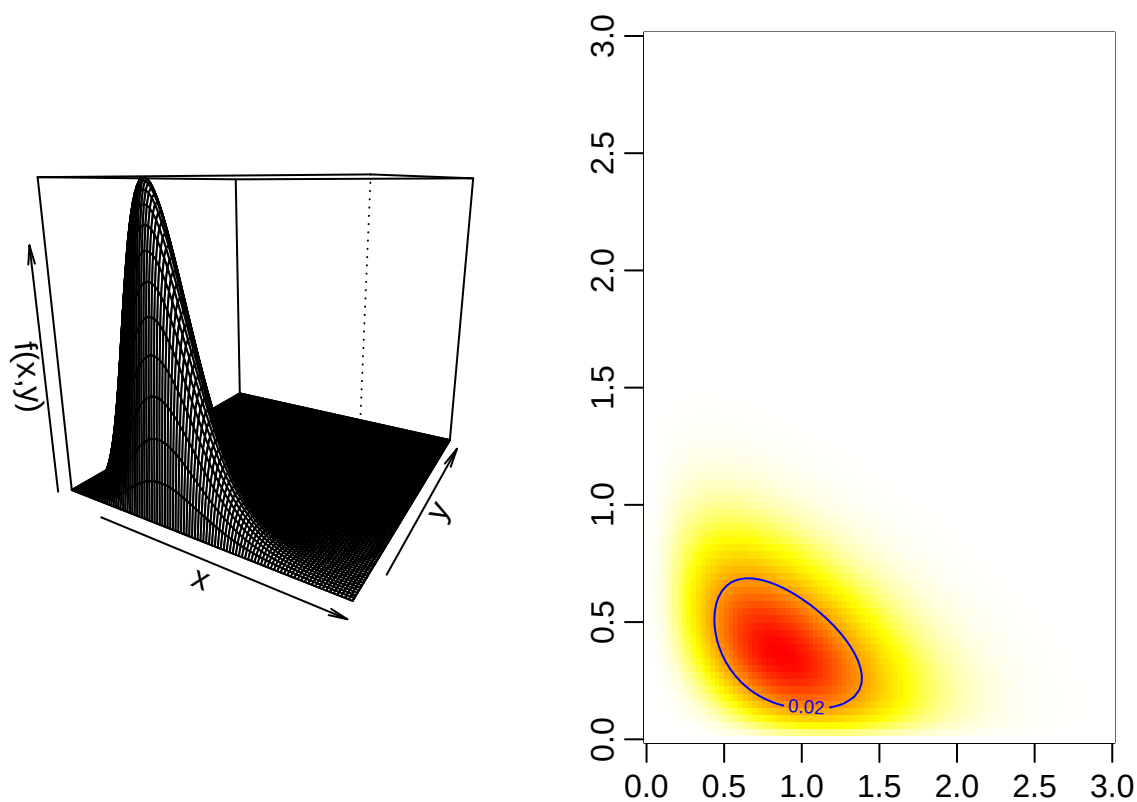

Figure 6: Visualization of $f(x, y)$ up to a constant c

Since the support of $f(x, y)$ is contained in that of bivariate chi-square distribution in which the two degrees of freedom for the two marginals are both 0.3, I consider to generate the random sample via Metropolis-Hastings (MH) with proposal q, the density of the aforementioned bivariate chi-square distribution. The degrees of freedom are chosen in hope that the looks of f and q are as similar as possible (see Figure **??**).

```
q = function(arg){
  x = arg[1]
  y = arg[2]
  return(dchisq(x, df = 0.3)*dchisq(y, df = 0.3))
}
quilt.plot(grid2[,1], grid2[,2], z, nx = 100, ny = 100, col = rev(heat.colors(1000)),
add.legend = F)
contour(grid1, grid1, matrix(z, 100, 100),
levels = round(quantile(z, seq(0.05,0.95,0.1)), 2), col = "blue", add = T)
z2 = apply(grid2, 1, q)
contour(grid1, grid1, matrix(z2, 100, 100),
levels = round(quantile(z2, seq(0.05,0.95,0.1)), 2), col = "red", add = T)
legend("topright", c("cf(x,y)", "q(x,y)"), lty = 1, col = c("blue", "red"), bty = "n")
```

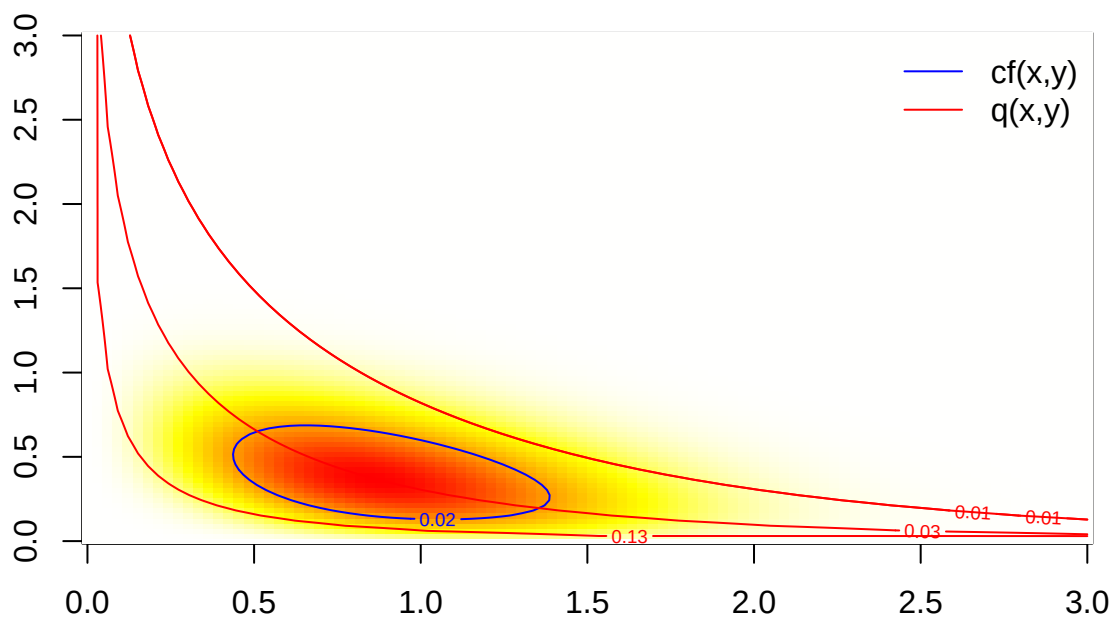

Figure 7: (#fig:qShape,)Visualization of $cf(x, y)$ and $q(x, y)$

Algorithm

See appendix

Implementation

```
mh = function(n){
  init = c(0.1, 0.1)
  x.m = matrix(NA, nrow = n, ncol = 2)
```

```
  x.m[1,] = init
  for(i in 2:n){
    u = rchisq(2, df = 0.3)
    r = (f(u)/f(x.m[i-1,]))*(q(x.m[i-1,])/q(u))
    if(runif(1)<min(1,r)){
      x.m[i,] = u
    }else{
      x.m[i,] = x.m[i-1,]
    }
  }
  return(x.m)
}
```

```
n = 10^5
x.m1 = mh(n)
x.m2 = mh(n)
x.m3 = mh(n)
x.m4 = mh(n)
mc1234 = mcmc.list(mcmc(x.m1), mcmc(x.m2), mcmc(x.m3), mcmc(x.m4))
dump(c("mc1234"), "midterm_2.Rdata")
```

The change of Gelman-Rubin stastic along with the length of the Markov chains is shown in Figure 8. It seems that the Markov chains are convergent to the target distribution after the length is greater 92000. Hence, keep only the last 8000 sample points for the 4 Markov chains, resulting in 4 shortened Markov chains.

```
gelman.plot(mc1234)
```

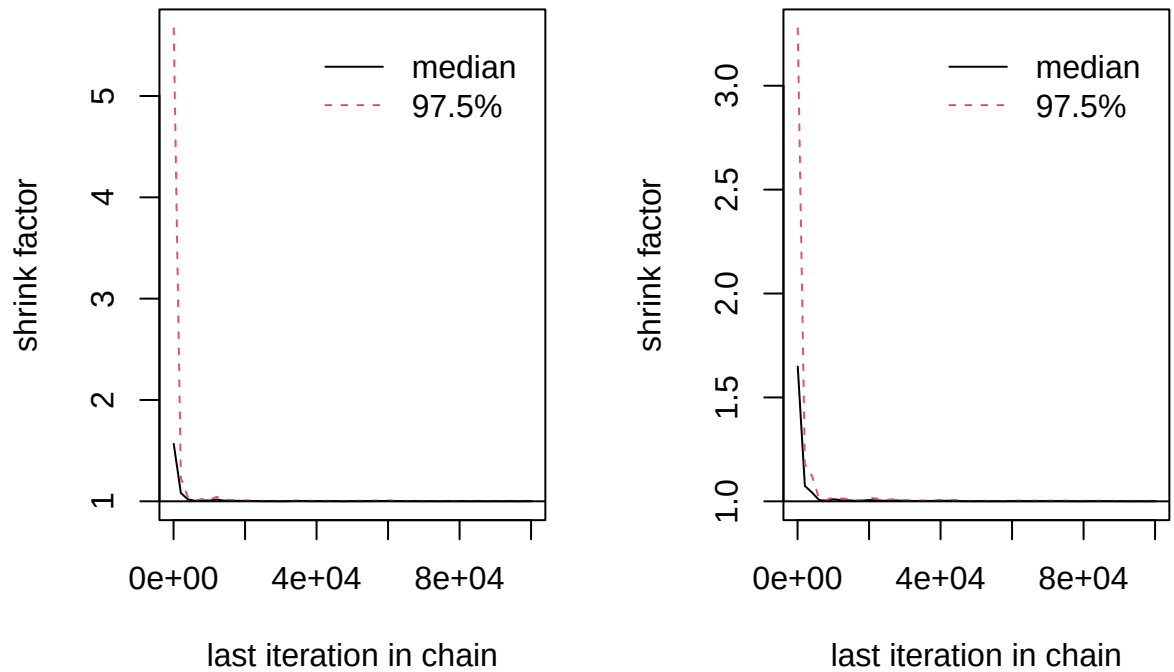

Figure 8: The change of Gelman-Rubin stastic along with the length of the Markov chains

After a few trials and errors, it turns out that by picking the sample points from each of the shortened Markov chains by every 32 ones, we can come up with an nearly i.i.d sample with size 1000, the final sample as required. Figure 9 shows the acf plots for the final sample. We can see that the autocorrelations of lag greater than or equal to 1 are not significant.

```
x.m = rbind(tail(mc1234[[1]], 8000)[32*(1:floor(8000)/32),],
            tail(mc1234[[2]], 8000)[32*(1:floor(8000)/32),],
            tail(mc1234[[3]], 8000)[32*(1:floor(8000)/32),],
            tail(mc1234[[4]], 8000)[32*(1:floor(8000)/32),])
x.m = x.m[sample(1000),]
acf(x.m)
```

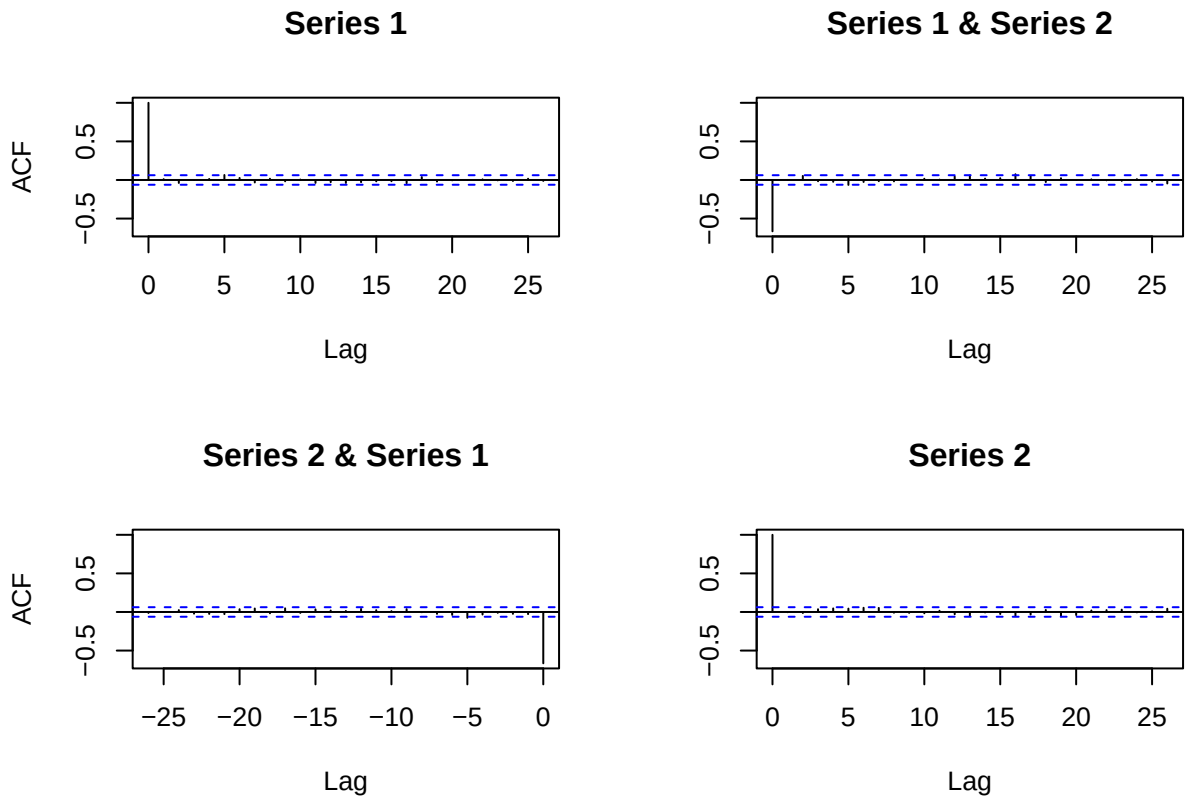

Figure 9: The acf plots for the final sample

Sampling result

Figure 10 shows the final sample by plotting the generated variables on the contour plot of $f(x, y)$. We can see that the distribution of the generated sample looks quite similar to $f(x, y)$.

```
grid1 = seq(0.0001, 3, length.out = 100)
grid2 = expand.grid(grid1, grid1)
z = apply(grid2, 1, f)
par(mfrow=c(1,1), mar=c(3,3,1,1), mgp=c(1.6,0.6,0))
quilt.plot(grid2[,1], grid2[,2], z, nx = 100, ny = 100, col = rev(heat.colors(1000)),
add.legend = F)
points(x.m, pch = 16)
contour(grid1, grid1, matrix(z, 100, 100),
levels = round(quantile(z, seq(0.05,0.95,0.1)), 2), col = "blue", add = T)
```

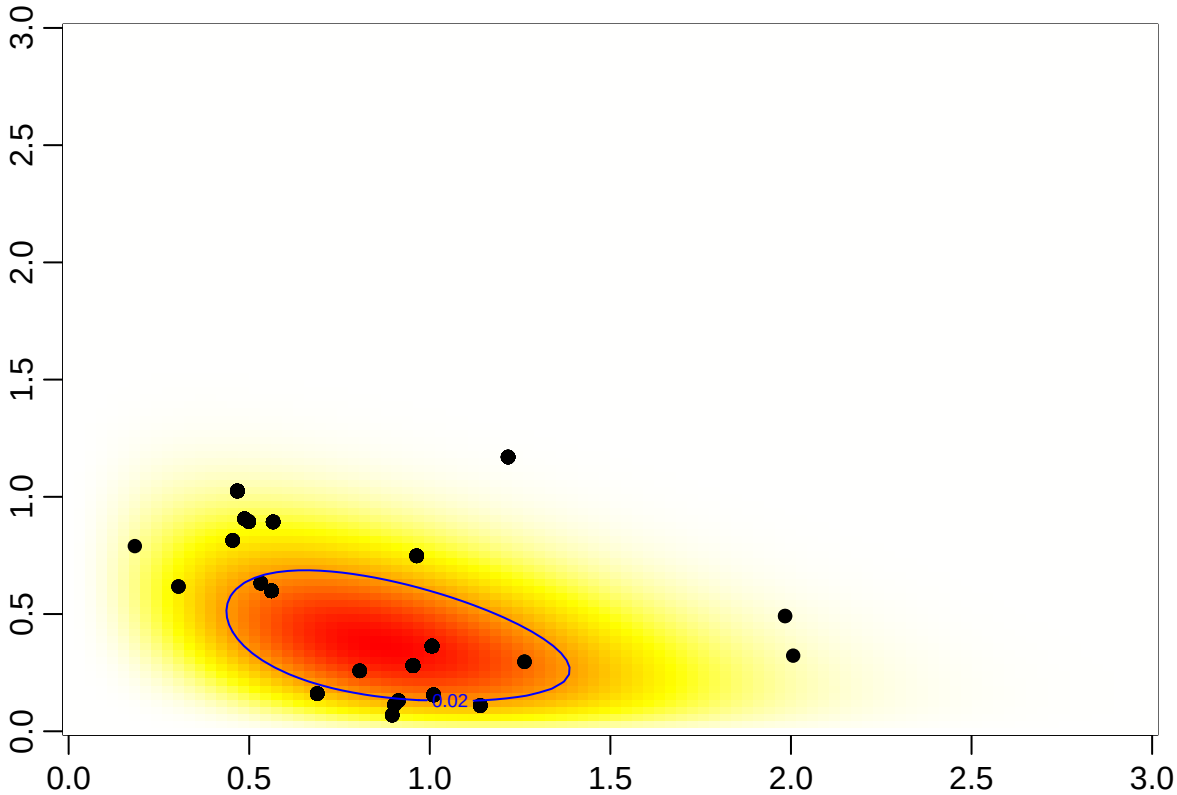

Figure 10: Sampling result in the view of $f(x, y)$

Figure 11 shows marginal distribution of the sample. We can see that the two marginal distributions looks quite similar to $f(x)$ and $f(y)$ respectively, where $f(x)$ and $f(y)$ are numerically obtained by the R function `cuhre`.

```
c = cuhre(g,  lowerLimit = c(0, 0), upperLimit = c(Inf, Inf))$integral
f.x = function(x) as.numeric(cuhre(function(y) {g(c(x,y))}/c, lowerLimit = 0, upperLimit = Inf)$integral

f.y = function(y) as.numeric(cuhre(function(x) {g(c(x,y))}/c, lowerLimit = 0, upperLimit = Inf)$integral

f.x <- Vectorize(f.x)
f.y <- Vectorize(f.y)

par(mfrow=c(1,2), mar=c(3,3,1,1), mgp=c(1.6,0.6,0))
hist(x.m[,1], 30, xlab = "y", main = "", probability = T)
curve(f.x, from = 0, to = 3, add = T, col = 2)
legend("topleft","f(x)", bty = "n", lty = 1, col =2)
hist(x.m[,2], 30, xlab = "y", main = "", probability = T)
curve(f.y, from = 0, to = 3, add = T, col = 2)
legend("topleft","f(y)", bty = "n", lty = 1, col = 2)
```

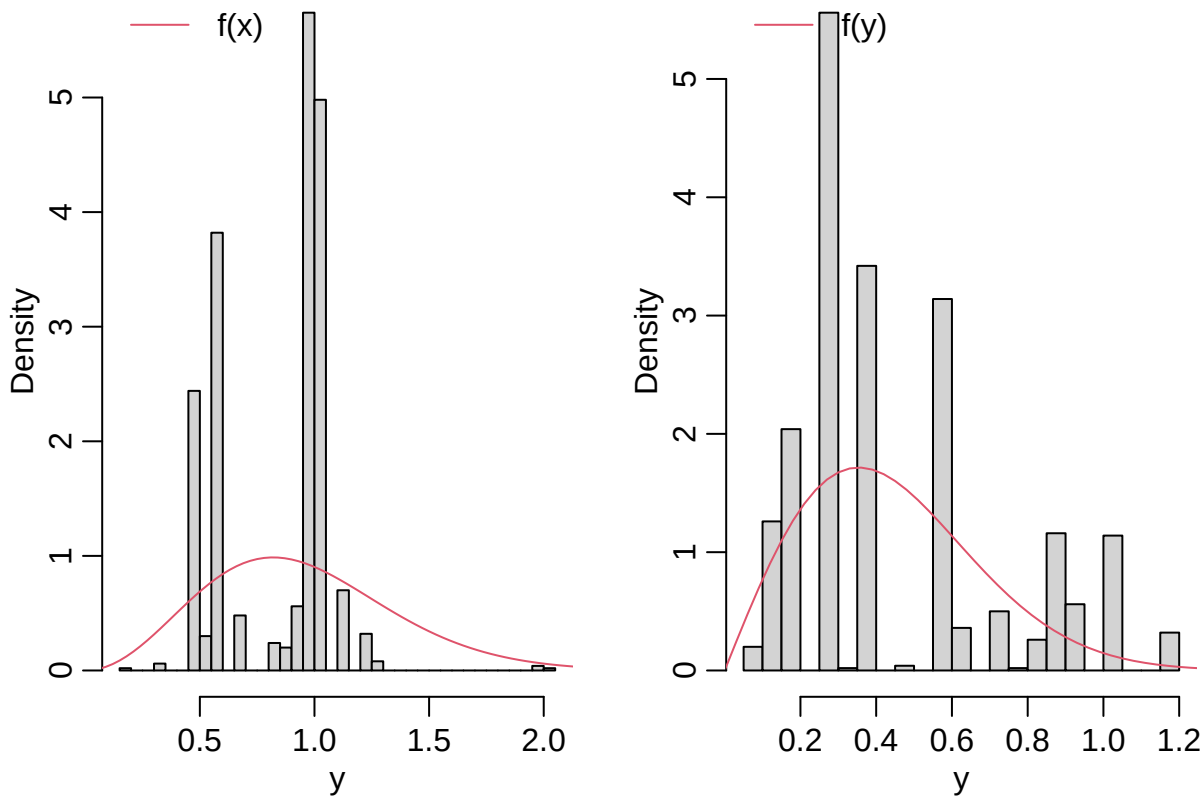

Figure 11: Sampling result in the view of $f(x)$ and $f(y)$

2.2 b.

For the algorithm and the formula of MC estimate and MC s.e., please see the appendix.

```
est = mean(apply(x.m, 1, g)/apply(x.m, 1, q))
se = sd(apply(x.m, 1, g)/apply(x.m, 1, q))/sqrt(1000)
c(est, se)
```

```
## [1] 0.77825827 0.01304349
```

It turns out the estimate is 0.7782583 and the s.e. is 0.0130435.